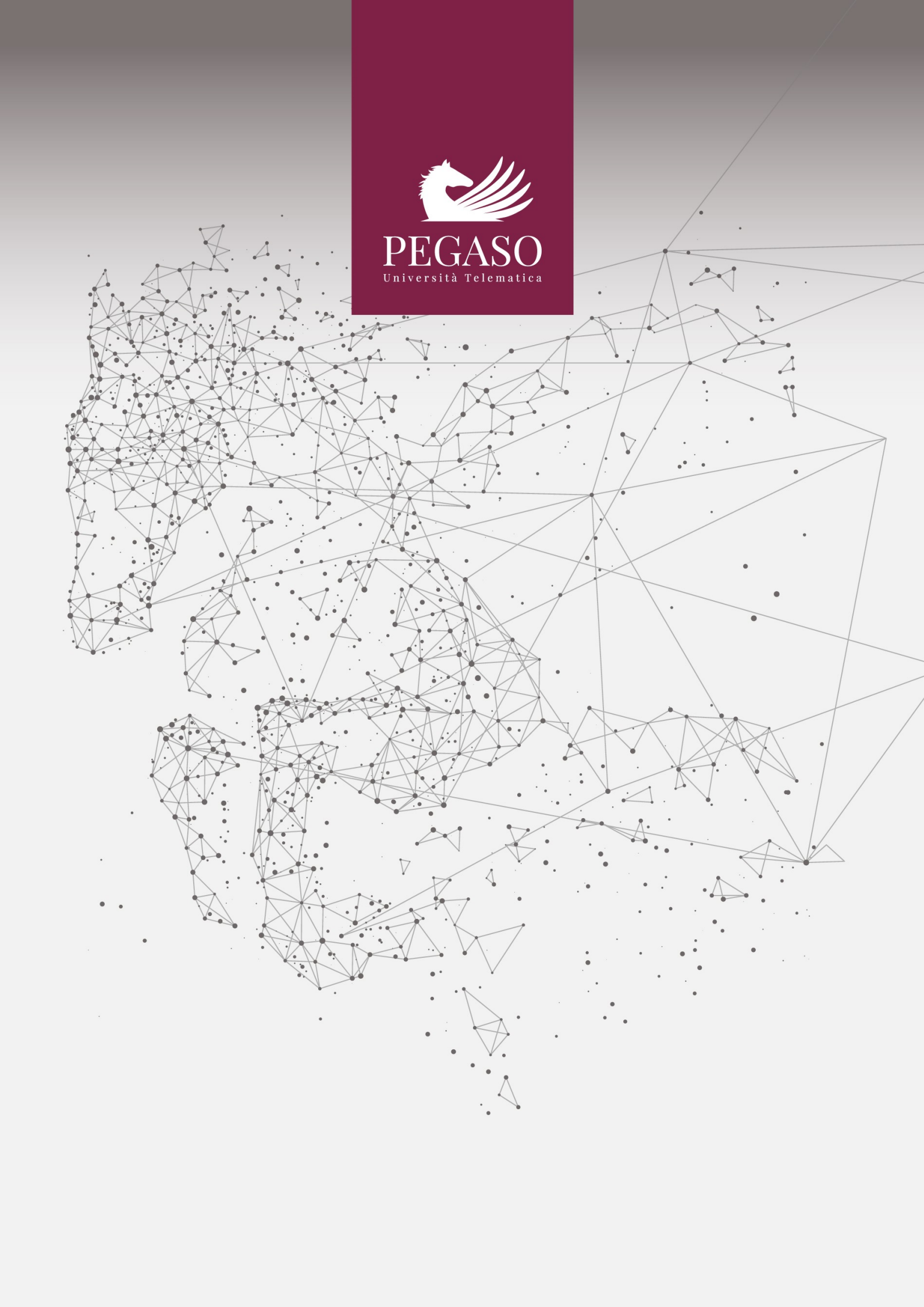

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. CONCETTO DI DIZIONARIO 4

3. OPERAZIONI SUI DIZIONARI 6

3.1 METODI PRINCIPALI DELL'OGGETTO DIZIONARIO 7

3.2 ESEMPIO DI PROGRAMMA SUI DIZIONARI: DIZIONARIO DI NOMI E NUMERI DI TELEFONO 12

4. CONCETTO DI SET 16

5. OPERAZIONI SUI SET 17

5.1 METODI PRINCIPALI DELL'OGGETTO SET 17

5.2 OPERAZIONI INSIEMISTICHE SUI SET 19

5.3 ESEMPI DI OPERAZIONI SUI SET 21

BIBLIOGRAFIA 22

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono due strutture dati molto importanti per il linguaggio di programmazione Python: i dizionari e i set. Per ciascuna delle due strutture dati viene fornita una panoramica introduttiva seguita da dettagli in merito a istruzioni, funzioni e metodi offerti da Python per l'utilizzo delle strutture stesse all'interno di un programma.

Infine, vengono presentati alcuni esempi di utilizzo delle operazioni possibili sulle due strutture dati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Concetto di dizionario

Un **dizionario** è un oggetto che memorizza una collezione di dati in modo diverso da una lista. Infatti, ciascun elemento del dizionario consiste di una **coppia chiave (key) - valore (value).**

La corrispondenza tra la chiave e il valore rappresenta un vero e proprio mapping della chiave sul valore.

La chiave, che rappresenta un vero e proprio indice per la ricerca del valore corrispondente, deve essere un oggetto immutabile. Un esempio tipico di dizionario potrebbe essere la rubrica di un cellulare in cui la chiave è il nome della persona e il valore è il numero telefonico associato alla persona stessa.

Esistono principalmente due formati per la creazione di un dizionario:

1. per creare un dizionario già popolato da elementi, il formato è:

```
dizionario = {chiave1:valore1, chiave2:valore2, ...}
```

2. per creare un dizionario vuoto, invece, il formato è:

```
dizionario = dict()
```

Un aspetto molto importante da considerare quando si opera con i dizionari è che gli elementi in un dizionario non sono ordinati (e non sono ordinabili in generale).

Proprio per la natura del dizionario ed, in particolare della chiave, è possibile ritrovare un valore dal dizionario data la chiave nel seguente modo:

```
dizionario[chiave]
```

Se la chiave tra parentesi quadre è una chiave che è presente nel dizionario viene restituito il valore associato, altrimenti viene sollevata un'**eccezione KeyError**.

Per prevenire le eccezioni KeyError, è possibile verificare se la chiave è presente o non è presente in un dizionario usando gli **operatori in** e **not in** come nell'esempio che segue:

```
if chiave in dizionario:
    print(dizionario[chiave])
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel caso in cui non vengano utilizzati tali operatori, è necessario gestire l'eccezione KeyError come nel seguente esempio:

```
try:
    print(dizionario[chiave])
except KeyError:
    print("Chiave ",chiave,"non presente nel dizionario")
```

I dizionari sono **oggetti mutabili** e per aggiungere una nuova coppia key-value si utilizza il seguente formato di istruzione:

```
dizionario[chiave] = valore
```

Di fondamentale importanza è rilevare che una chiave può essere presente solo una volta nel dizionario, mentre lo stesso valore, può essere presente più di una volta associato a chiavi diverse. Se la chiave esiste già nel dizionario, nel caso in cui si utilizzi l'istruzione precedente, il valore associato alla chiave verrà modificato.

I valori associati alle chiavi possono essere di qualsiasi tipo e non devono essere omogenei, cioè non devono essere tutti dello stesso tipo di dato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Operazioni sui dizionari

Nella trattazione dei dizionari, risulta di fondamentale importanza conoscere tutte le operazioni (istruzioni, funzioni e metodi) che Python mette a disposizione.

Per cancellare una coppia chiave-valore dal dizionario è necessario usare l'**istruzione del** nel seguente modo:

```
del dizionario[chiave]
```

Se la chiave non è nel dizionario, viene sollevata un'eccezione KeyError. Anche in questo caso è possibile utilizzare l'**operatore in** per verificare la presenza della chiave nel dizionario e procedere con la cancellazione della coppia chiave-valore solo in caso di presenza della chiave stessa:

```
if chiave in dizionario:
    del dizionario[chiave]
```

Segue un esempio di codice per la gestione dell'eccezione KeyError nel caso in cui si decida di non utilizzare l'operatore in o not in:

```
try:
    del dizionario[chiave]
except KeyError:
    print("Chiave ",chiave,"non presente nel dizionario")
```

Per conoscere il numero di elementi (coppie chiave-valore) in un dizionario è possibile utilizzare la **funzione len**() come mostrato nel seguente esempio:

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
print(len(dizionario))
```

Il risultato dell'elaborazione del codice precedente che verrà visualizzato a schermo sarà 3.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per iterare sul dizionario al fine di accedere agli elementi presenti è possibile utilizzare un ciclo for. Nell'esempio seguente, si riporta il codice per stampare tutte le chiavi del dizionario:

```
for chiave in dizionario:
    print(chiave)
```

Nell'esempio seguente, invece, si riporta il codice per stampare tutte le coppie chiave-valore presenti nel dizionario:

```
for chiave in dizionario:
    print(chiave, dizionario[chiave])
```

3.1 Metodi principali dell'oggetto dizionario

Si presentano ora i metodi principali dell'oggetto dizionario offerti da Python. Per ciascun metodo verrà fornita una breve spiegazione, un esempio esplicativo di codice sorgente e il risultato dell'elaborazione visualizzato a schermo.

- **Metodo clear()**

Il metodo **clear**() cancella tutte le coppie chiave-valore lasciando il dizionario completamente vuoto.

Si riporta il codice sorgente di esempio in cui si inizializza e si stampa un dizionario con tre coppie chiave-valore. Successivamente, si chiama il metodo clear() senza argomenti e si ristampa il dizionario che, questa volta, sarà completamente vuoto:

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
print("Dizionario inizializzato:",dizionario)
dizionario.clear()
print("Dizionario dopo chiamata a clear():",dizionario)
```

Risultato dell'elaborazione visualizzato a schermo:

```
Dizionario inizializzato: {'Pasquale': '1234'
, 'Maria': '5678', 'Giuseppe': '90123'}
Dizionario dopo chiamata a clear(): {}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Metodo get()**

Il metodo **get**() recupera un valore associato ad una specifica chiave del dizionario utilizzando il seguente formato:

```
dizionario.get(chiave,default)
```

Il metodo get() si comporta come l'operatore [ ] (es. dizionario[chiave]), ma se la chiave non viene trovata restituisce il valore **default** e non solleva eccezione KeyError.

Si riporta il codice sorgente di esempio in cui si inizializza un dizionario con tre coppie chiave-valore e, successivamente, si recupera una coppia chiave-valore sulla base di una chiave presente nel dizionario ("Pasquale") ed una chiave non presente nel dizionario ("Antonio"). Per quest'ultima verrà stampato il valore di default ("Non trovato"):

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
print("Pasquale:",dizionario.get("Pasquale","Non trovato"))
print("Antonio:",dizionario.get("Antonio","Non trovato"))
```

Risultato dell'elaborazione visualizzato a schermo:

```
Pasquale: 1234
Antonio: Non trovato
```

- **Metodo items()**

Il metodo **items**() restituisce tutte le chiavi e i valori associati del dizionario (in pratica corrisponde ad una vista del dizionario). Il formato da utilizzare è il seguente:

```
t = dizionario.items()
```

In particolare, ogni elemento dell'oggetto t è una tupla che contiene una coppia chiave-valore associato. Si riporta il codice sorgente di esempio in cui si stampa il numero di elementi compresi nel dizionario mostrato in precedenza ("Pasquale", "Maria", "Giuseppe") e l'oggetto t restituito dal metodo items():

```
print("La variabile t ha dimensione",len(t))
print(t)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Risultato dell'elaborazione visualizzato a schermo:

```
La variabile t ha dimensione 3
dict_items([('Pasquale', '1234'), ('Maria', '
5678'), ('Giuseppe', '90123')])
```

Per iterare sulle tuple restituite dal metodo items(), è possibile utilizzare un ciclo for come nel codice sorgente seguente:

```
t = dizionario.items()
for tupla in t:
    print(tupla)
```

Il risultato dell'elaborazione visualizzato a schermo sarà, dunque:

```
('Pasquale', '1234')
('Maria', '5678')
('Giuseppe', '90123')
```

- **Metodo keys()**

Sulla falsariga del metodo items(), il metodo **keys**() restituisce tutte le chiavi del dizionario in una sequenza (e non anche i valori associati alle chiavi). Il formato da utilizzare è il seguente:

```
k = dizionario.keys()
```

Si riporta il codice sorgente di esempio in cui si richiama il metodo keys() sul solito dizionario di esempio e, successivamente, si stampa il risultato k restituito dal metodo keys() e si itera su tale oggetto per stampare una chiave alla volta:

```
k = dizionario.keys()
print(k)
for chiave in k:
    print(chiave)
```

Il risultato dell'elaborazione visualizzato a schermo sarà, dunque:

```
dict_keys(['Pasquale', 'Maria', 'Giuseppe'])
Pasquale
Maria
Giuseppe
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Metodo values()**

In contrapposizione al metodo keys(), il metodo **values**() restituisce tutti i valori del dizionario in una sequenza (e non anche le chiavi ad essi associati). Il formato da utilizzare è il seguente:

```
valori = dizionario.values()
```

Si riporta il codice sorgente di esempio in cui si inizializza il dizionario e si richiama il metodo values() sul dizionario stesso. Successivamente, si stampa il risultato valori restituito dal metodo values() e si itera su tale oggetto per stampare un valore alla volta:

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
valori = dizionario.values()
print(valori)
for val in valori:
    print(val)
```

Il risultato dell'elaborazione visualizzato a schermo sarà, dunque:

```
dict_values(['1234', '5678', '90123'])
1234
5678
90123
```

- **Metodo pop()**

Il metodo **pop**() restituisce il valore associato ad una specifica chiave e, allo stesso tempo, rimuove la coppia chiave-valore dal dizionario. Il metodo pop() permette di evitare l'eccezione KeyError poiché se la chiave non esiste restituisce il valore di default. Il formato da utilizzare per richiamare il metodo pop() è il seguente:

```
valore = dizionario.pop(chiave, default)
```

Si riporta il codice sorgente di esempio in cui si inizializza il dizionario e la variabile chiave a "Pasquale". Successivamente, si richiama il metodo pop sul dizionario utilizzando come parametri la chiave "Pasquale" e il valore di default "Non Valido".

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, si stampa sia la coppia chiave-valore, sia il dizionario a seguito della chiamata a pop() che risulterà composto da due coppie e non più da tre:

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
chiave = "Pasquale"
valore = dizionario.pop(chiave,'Non valido')
print("La coppia (",chiave,"-",valore,") è stata estratta ed eliminata")
print("Il dizionario ora è:", dizionario)
```

Il risultato dell'elaborazione visualizzato a schermo sarà, dunque:

```
La coppia ( Pasquale - 1234 ) è stata
estratta ed eliminata
Il dizionario ora è: {'Maria': '5678',
'Giuseppe': '90123'}
```

- **Metodo popitem()**

Il metodo **popitem**() restituisce come il metodo pop() una coppia chiave-valore e la rimuove dal dizionario, ma la scelta della coppia chiave valore avviene in modo casuale: la coppia è, infatti, scelta a caso. A titolo di esempio, si pensi a quanto possa essere utile tale metodo per simulare l'estrazione di carte in un mazzo di carte rappresentato da un dizionario {"Asso di picche":1, "2 di picche":2, …}.

La coppia chiave-valore viene restituita come tupla.

Si riporta il codice sorgente di esempio in cui si inizializza il dizionario e si richiama il metodo popitem() che restituirà una coppia scelta a caso che verrà visualizzata a schermo con la funzione print():

```
dizionario = {'Pasquale':'1234', \
              'Maria':'5678', 'Giuseppe':'90123'}
coppia = dizionario.popitem()
print("Coppia chiave-valore estratta ed eliminata è:", coppia)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione visualizzato a schermo sarà, dunque, il seguente in cui la coppia estratta casualmente è "Giuseppe":"90123":

```
Coppia chiave-valore estratta ed eliminata è:
('Giuseppe', '90123')
```

3.2 Esempio di programma sui dizionari: dizionario di nomi e numeri di telefono

Al fine di mostrare l'utilità e l'applicazione dei metodi dei dizionari appena descritti, si propone un esercizio di creazione e gestione di un dizionario di nomi e numeri di telefono che simulerà la rubrica di un telefono cellulare. La chiave del dizionario sarà rappresentata dal nome di una persona, mentre il valore associato alla chiave sarà il numero di telefono della persona stessa.

Si ricorda che non possono esistere in un dizionario due chiavi uguali. Ne consegue che i nomi delle persone dovranno essere distinguibili ad esempio aggiungendo, se necessario, il cognome o altri elementi di distinzione. Ciò non rappresenta un limite poiché anche nella gestione della propria rubrica telefonica del cellulare sarebbe complesso distinguere il numero corretto di due persone che hanno lo stesso nominativo.

Il programma organizzato in funzioni, che partirà da un dizionario vuoto, sarà realizzato con l'utilizzo di un menu al fine di consentire le seguenti scelte:

1. cercare un numero di telefono;
2. aggiungere un numero di telefono;
3. modificare un numero di telefono;
4. eliminare un numero di telefono;
5. uscire dal programma.

Si riporta di seguito il codice sorgente che imposta il valore delle costanti globali di scelta (corrispondenti alle voci del menu) e la definizione della funzione main(). Nella funzione main, si inizializza un dizionario vuoto denominato numeri. Viene poi inizializzata la variabile di scelta con un

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

valore non ricompreso tra le possibili scelte del menu (in questo caso 0) al fine di entrare nel ciclo while e procedere con la scelta effettiva dell'operazione da eseguire.

Nel ciclo while viene richiamata la funzione get_scelta_menu() che restituirà il numero di scelta effettuata. La catena di if-elif permetterà di chiamare la funzione corrispondente alla scelta effettuata.

Il ciclo while terminerà solo quando la scelta effettuata sarà uguale al valore della costante ESCI che è pari a 5.

```
CERCA = 1
AGGIUNGI = 2
CAMBIA = 3
CANCELLA = 4
ESCI = 5


def main():
    numeri = dict()  #rubrica vuota

    scelta = 0       #scelta volutamente non valida

    while scelta!=ESCI:
        scelta = get_scelta_menu()

        if scelta == CERCA:
            cercare(numeri)
        elif scelta == AGGIUNGI:
            aggiungere(numeri)
        elif scelta == CAMBIA:
            modificare(numeri)
        elif scelta == CANCELLA:
            cancellare(numeri)
```

Si riporta ora il codice sorgente della funzione get_scelta_menu() che stampa a schermo il menu con tutte le opzioni possibili e, attraverso un ciclo di validazione dell'input, permette l'input di un valore di scelta compreso tra 1 e 5. Al termine del ciclo di validazione restituisce il valore della variabile scelta.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
#funzione che visualizza scelte possibili del menu
def get_scelta_menu():
    print()
    print("Rubrica telefonica")
    print("--------------------")
    print("1. Cerca numero di telefono")
    print("2. Aggiungi numero di telefono")
    print("3. Modifica numero di telefono")
    print("4. Elimina numero di telefono")
    print("5. Esci dal programma")
    print()
    scelta = int(input("Inserire una scelta: "))

    while scelta<CERCA or scelta>ESCI:
        scelta = int(input("Inserire una scelta valida: "))

    return scelta
```

Si passa, dunque, al codice sorgente delle funzioni cercare() e aggiungere(). La funzione cercare() accetta come parametro il dizionario dei numeri di cellulare. Nel corpo della funzione richiede un nome da ricercare e stampa il numero di telefono utilizzando il metodo get() del dizionario. Nel caso in cui il nome della persona non sia nel dizionario, a schermo verrà visualizzato il valore di default "Non trovato".

Anche la funzione aggiungere() accetta come parametro il dizionario dei numeri di cellulare. Nel corpo della funzione richiede il nome della persona da aggiungere. In seguito, verifica che il nome non sia già presente nel dizionario: in questo caso richiede anche il numero di telefono e lo aggiunge al dizionario. Se il nome, invece, è già presente nel dizionario, viene visualizzato a schermo il seguente messaggio "La voce esiste già".

```
#funzione di ricerca di un numero telefonico
def cercare(numeri):
    nome = input("Inserire un nome: ")
    print(numeri.get(nome, "Non trovato"))


#funzione di aggiunta di un numero telefonico
def aggiungere(numeri):
    nome = input("Inserire un nome: ")

    if nome not in numeri:
        num = input("Inserire un numero di telefono: ")
        numeri[nome]=num
    else:
        print("La voce esiste già")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, si riporta il codice sorgente delle funzioni modificare() e cancellare(). La funzione modificare() accetta come parametro il dizionario dei numeri di cellulare. Nel corpo della funzione richiede il nome da modificare: se il nome è presente nel dizionario richiede il nuovo numero di telefono e lo aggiorna. In caso contrario, viene visualizzato il seguente messaggio "Nome non presente in rubrica".

La funzione cancellare() accetta come parametro il dizionario dei numeri di cellulare. Nel corpo della funzione richiede il nome da eliminare: se il nome è presente nel dizionario cancella la coppia nome-numero di telefono. In caso contrario, viene visualizzato il seguente messaggio "Nome non presente in rubrica".

```
#funzione di modifica di un numero telefonico
def modificare(numeri):
    nome = input("Inserire un nome: ")

    if nome in numeri:
        num = input("Inserire il nuovo numero di telefono: ")
        numeri[nome]=num
    else:
        print("Nome non presente in rubrica")


#funzione di eliminazione di un numero telefonico
def cancellare(numeri):
    nome = input("Inserire un nome: ")

    if nome in numeri:
        del numeri[nome]
    else:
        print("Nome non presente in rubrica")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Concetto di set

Un'altra struttura dati molto utilizzata in Python è rappresentata dagli insiemi, i **set**. Un set è un oggetto che memorizza una collezione di dati nello stesso modo di un insieme matematico. Tutti gli elementi devono essere **unici**, cioè presenti solo una volta nel dizionario. Il set non è ordinato e gli elementi al suo interno possono essere di tipi di dati diversi, quindi non devono necessariamente omogenei.

Esistono principalmente due formati per la creazione di un set:

1. per creare un set vuoto il formato è:

```
insieme = set()
```

2. per creare un set già popolato da elementi il formato è il seguente in cui argomento deve essere un oggetto che contiene elementi iterabili come liste, stringhe o tuple:

```
insieme = set(argomento)
```

Se argomento è una stringa, l'insieme sarà formato dai caratteri (non ripetuti) della stringa come nell'esempio seguente:

```
insieme = set("abba")
print(insieme)
```

che visualizzerà a schermo un set formato solo da due lettere a e b:

```
{'b', 'a'}
```

Per avere un insieme di stringhe, le stringhe devono essere inserite prima in una lista. Se argomento contiene duplicati, nel set verrà inserita una sola occorrenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Operazioni sui set

I **set** sono oggetti modificabili. Anche per i set, Python offre diverse istruzioni, funzioni e metodi per la creazione e la gestione.

Anche per i set, la funzione **len**() permette di conoscere il numero di elementi del set. Nel codice sorgente che segue, viene inizializzata una lista che viene, a sua volta, utilizzata come argomento per la funzione set() di creazione di un insieme. Viene, poi, visualizzato a schermo il numero degli elementi dell'insieme e gli elementi in esso contenuti:

```
lista = ["alfa", "beta", "gamma", "delta"]

insieme = set(lista)
print("L'insieme contiene", len(insieme), "elementi")
print(insieme)
```

Il risultato dell'elaborazione mediante la visualizzazione a schermo è:

```
L'insieme contiene 4 elementi
{'delta', 'gamma', 'alfa', 'beta'}
```

5.1 Metodi principali dell'oggetto set

Si presentano ora i metodi principali dell'oggetto set offerti da Python. Per ciascun metodo verrà fornita una breve spiegazione, un esempio esplicativo di codice sorgente e il risultato dell'elaborazione visualizzato a schermo.

- **Metodo add()**

Il metodo **add**() aggiunge un elemento al set. Si riporta il codice sorgente di esempio in cui si inizializza il set e si richiama il metodo add() passandogli la stringa "delta". Infine, verrà visualizzato il set a schermo con la funzione print():

```
insieme = set(["alfa", "beta", "gamma"])
insieme.add("delta")
print(insieme)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Si riporta di seguito il risultato dell'elaborazione. Si noti che gli elementi dell'insieme non sono ordinati né in ordine alfabetico, né in ordine di inserimento.

```
{'gamma', 'delta', 'alfa', 'beta'}
```

- **Metodo update()**

Il metodo **update**(), al contrario del metodo add(), aggiunge un gruppo di oggetti iterabili ad un set (liste, stringhe e tuple). Si riporta il codice sorgente di esempio in cui si inizializza una lista e si crea un insieme vuoto. Si richiama, quindi, il metodo update() passandogli la lista. Infine, verrà visualizzato il set a schermo con la funzione print():

```
lista = ["alfa", "beta", "gamma"]
insieme = set()
insieme.update(lista)
print(insieme)
```

Il risultato dell'elaborazione sarà dunque:

```
{'gamma', 'alfa', 'beta'}
```

- **Metodi remove()** e **discard()**

I metodi **remove**() e **discard**() rimuovono l'elemento specificato (come parametro) dal set.

La differenza tra i due metodi è che remove(), in caso di non presenza dell'elemento nel set, solleva un'eccezione KeyError. Al contrario, discard() non solleva eccezioni. Si riporta il codice sorgente di esempio in cui si crea un insieme vuoto e si richiamano i metodi remove() e discard() passando loro l'elemento "alfa".

L'obiettivo del codice è quello di dimostrare la differenza tra le due funzioni in termini di eccezioni:

```
insieme = set()
print("rimuovo alfa con discard")
insieme.discard("alfa")
print("rimuovo alfa con remove")
insieme.remove("alfa")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come previsto, la rimozione con il metodo discard() di un elemento su insieme vuoto non solleva eccezioni, mentre la stessa operazione con il metodo remove() solleva l'eccezione KeyError:

```
rimuovo alfa con discard
rimuovo alfa con remove
Traceback (most recent call last):
  File "C:\Users\     d\AppData\Local\Programs
\Python             e.py", line 6, in <modul
e>
    insieme.remove("alfa")
KeyError: 'alfa'
```

- **Metodo clear()**

  Il metodo **clear**(), come per i dizionari, cancella tutti gli elementi del dizionario stesso.

Ai metodi presentati, è opportuno ricordare che con i set possono essere utilizzati il **ciclo for** che considera ogni elemento del set solo una volta e gli **operatori in** e **not in** per verificare che l'elemento appartenga o meno al set.

5.2 Operazioni insiemistiche sui set

Essendo i set una modellazione del concetto matematico di insieme, Python offre anche altre operazioni prettamente insiemistiche.

Per l'**unione** tra due set che permette di ottenere un nuovo set contenente gli elementi di entrambi i set di partenza è possibile utilizzare o il metodo union() o l'operatore | come segue:

set1.union(set2)

set1 | set2

Per l'**intersezione** tra due set che permette di ottenere un nuovo set contenente l'intersezione dei due set di partenza è possibile utilizzare o il metodo intersection() o l'operatore & come segue:

set1.intersection(set2)

set1 & set2

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per la **differenza** tra due set che permette di ottenere un nuovo set contenente gli elementi del primo set di partenza che non appartengono al secondo è possibile utilizzare o il metodo difference() o l'operatore - come segue:

set1.difference(set2)

set1 - set2

Per la **differenza simmetrica** tra due set (per la rappresentazione grafica si veda Figura 1) che permette di ottenere un nuovo set contenente gli elementi non condivisi tra i due set di partenza è possibile utilizzare o il metodo symmetric_difference() o l'operatore ^ come segue:

set1.symmetric_difference(set2)

set1 ^ set2

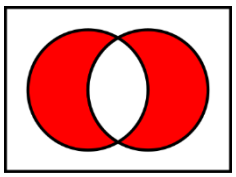


Figura 1. Rappresentazione grafica della differenza simmetrica tra due insiemi

Per la **verifica di sottoinsieme** tra due set per cui si verifica (restituisce True o False) che tutti gli elementi del primo set di partenza siano inclusi anche nel secondo set è possibile utilizzare o il metodo issubset() o l'operatore <= come segue:

set1.issubset(set2)

set1 <= set2

Per la **verifica di superinsieme** tra due set per cui si verifica (restituisce True o False) che il primo set di partenza contenga tutti gli elementi del secondo set è possibile utilizzare o il metodo issuperset() o l'operatore >= come segue:

set1.issuperset(set2)

set1 >= set2

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5.3 Esempi di operazioni sui set

Si riporta ora un breve codice sorgente di esempio per illustrare le operazioni sui set appena viste. In particolare, si inizializzano tre set. Successivamente, si stampano i risultati delle operazioni di:

- unione tra set1 e set2;
- intersezione tra set1 e set2;
- differenza tra set1 e set2;
- differenza simmetrica tra set1 e set2;
- verifica di sottoinsieme tra set3 e set2;
- verifica di superinsieme tra set1 e set3.

```
set1 = set(['a','b','c'])
set2 = set(['c','d','e'])
set3 = set(['c'])

print("Unione set1 e set2", set1|set2)
print("Intersezione set1 e set2", set1&set2)
print("Differenza set1 e set2", set1-set2)
print("Differenza simmetrica set1 e set2", set1^set2)
print("set3 è sottoinsieme di set2?", set3 <= set2)
print("set1 è superinsieme di set3?", set1 >= set3)
```

Il risultato dell'elaborazione è il seguente:

```
Unione set1 e set2 {'a', 'e', 'b', 'd', 'c'}
Intersezione set1 e set2 {'c'}
Differenza set1 e set2 {'b', 'a'}
Differenza simmetrica set1 e set2 {'d', 'a', 'e', 'b'}
set3 è sottoinsieme di set2? True
set1 è superinsieme di set3? True
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).